



هوش مصنوعی

بهار ۱۴۰۵

استاد: ماری اوریاد

گردآورندگان: رادین جریره، سید محمدرضا جوادی، کیارش شیخی،
امیرپارسا صفری، کیمیا علوی، علی مقدسی

مهلت ارسال: ۱۱ اردیبهشت

جستجوی ناآگاهانه، آگاهانه و محلی

تمرین اول

- مهلت ارسال پاسخ تا ساعت ۲۳:۵۹ روز مشخص شده است.
- در طول ترم امکان ارسال با تاخیر پاسخ همه‌ی تمارین تا سقف ۴ روز و در مجموع ۱۰ روز، وجود دارد. پس از گذشت این مدت، پاسخ‌های ارسال‌شده پذیرفته نخواهند بود. همچنین، به ازای هر ساعت تأخیر غیر مجاز ۰.۵ درصد از نمره تمرین به صورت ساعتی کسر خواهد شد.
- همکاری و هم‌فکری شما در انجام تمرین مانعی ندارد اما پاسخ‌های ارسال‌شده هر کس حتماً باید توسط خود او نوشته شده باشد.
- در صورت هم‌فکری و یا استفاده از هر منابع خارج درسی، نام هم‌فکران و آدرس منابع مورد استفاده برای حل سوال مورد نظر را ذکر کنید.
- لطفاً تصویری واضح از پاسخ سوالات نظری بارگذاری کنید. در غیر این صورت پاسخ شما تصحیح نخواهد شد.
- پاسخنامه سوالات را در سامانه کوئرا و به فرمت `AI_HW1_<STUDENT_ID>` بارگزاری کنید.

سوالات نظری (۱۰۰ نمره)

۱. (۲۰ نمره) درستی و نادرستی عبارات زیر را با ذکر دلیل مشخص کنید.
 - (آ) پیچیدگی فضایی الگوریتم DFS در جستجوی گراف، همچنان در اردر خطی $O(bm)$ است، زیرا در هر لحظه فقط گره‌های مسیر فعلی در حافظه قرار دارند.
 - (ب) اگر h_1 و h_2 دو تابع اکتشافی^۱ یک‌نوا^۲ باشند آنگاه $\frac{h_1+h_2}{2}$ نیز یک‌نوا است.
 - (ج) اگر h_1 یک تابع یک‌نوا باشد و c یک عدد ثابت بیشتر از ۱ باشد آنگاه $h_2 = c \times h_1$ نیز یک‌نوا است.
 - (د) جستجوی A^* در حالت درختی با یک تابع اکتشافی قابل قبول، همواره بهینه است.
 - (ه) با کاهش دما، عملکرد الگوریتم Simulated Annealing به عملکرد الگوریتم Local Beam Search با $k = 1$ میل می‌کند.
 - (و) الگوریتم IDS در بدترین حالت از نظر تعداد گسترش‌ها دقیقاً به اندازه BFS گره باز می‌کند.
 - (ز) اگر کوچک‌ترین هزینه یال برابر صفر باشد (یعنی یال‌هایی با هزینه ۰ وجود داشته باشد)، الگوریتم UCS ممکن است حتی در صورت وجود پاسخ با هزینه متناهی، خاتمه نیابد.
 - (ح) اگر h_1 و h_2 یک‌نوا باشند، آنگاه $\min(h_1, h_2)$ نیز یک‌نوا است.
 - (ط) در Local Beam Search با فرض محدود بودن زمان و حافظه، افزایش K لزوماً منجر به افزایش احتمال یافتن استیت هدف می‌شود.
 - (ی) اگر با شروع از وضعیت S ، الگوریتم Hill Climbing به وضعیت هدف G برسد، آنگاه الگوریتم Local Beam Search با $K > 1$ که S جزئی از جمعیت اولیه‌اش می‌باشد نیز لزوماً به وضعیت G می‌رسد.

^۱ heuristic
^۲ consistent

حل.

(آ) **نادرست.** در جستجوی درختی DFS، اگر عمق بیشینه برابر m و عامل انشعاب برابر b باشد، پیچیدگی فضایی معمولاً از مرتبه‌ی

$$O(bm)$$

است؛ زیرا الگوریتم در هر لحظه مسیر فعلی و تعدادی از فرزندانِ گره‌های روی مسیر را در حافظه نگه می‌دارد.

اما در جستجوی گراف، برای جلوگیری از بازدید دوباره‌ی حالت‌ها، معمولاً یک مجموعه‌ی explored نگه داشته می‌شود. این مجموعه می‌تواند در بدترین حالت شامل تعداد بسیار زیادی از حالت‌های تولید شده باشد.

بنابراین در حالت گراف، حافظه فقط شامل مسیر فعلی نیست. در بدترین حالت، حافظه می‌تواند متناسب با تعداد حالت‌های دیده شده باشد، یعنی چیزی در حد

$$O(|V|)$$

یا اگر بر حسب b و m بیان کنیم، در بدترین حالت:

$$O(b^m).$$

پس دلیل عبارت، یعنی در هر لحظه فقط گره‌های مسیر فعلی در حافظه قرار دارند، برای DFS در حالت غیر درخت درست نیست.

(ب) **درست.** فرض کنید h_1 و h_2 هر دو یکنوا باشند. تعریف یکنوا بودن این است که برای هر یال از گره n به گره n' با هزینه‌ی $c(n, n')$ داشته باشیم:

$$h(n) \leq c(n, n') + h(n').$$

چون h_1 یکنوا است:

$$h_1(n) \leq c(n, n') + h_1(n').$$

و چون h_2 نیز یکنوا است:

$$h_2(n) \leq c(n, n') + h_2(n').$$

دو نامساوی را با هم جمع می‌کنیم:

$$h_1(n) + h_2(n) \leq 2c(n, n') + h_1(n') + h_2(n').$$

حال دو طرف را بر ۲ تقسیم می‌کنیم:

$$\frac{h_1(n) + h_2(n)}{2} \leq c(n, n') + \frac{h_1(n') + h_2(n')}{2}.$$

اگر تعریف کنیم:

$$h(n) = \frac{h_1(n) + h_2(n)}{2},$$

آنگاه داریم:

$$h(n) \leq c(n, n') + h(n').$$

پس h نیز یکنوا است.

(ج) **نادرست.** برای رد عبارت، یک مثال نقض کافی می‌زنیم:
فرض کنید فقط دو گره داریم:

$$A \rightarrow G$$

که G گره هدف است و هزینه‌ی یال برابر است با:

$$c(A, G) = 1.$$

تابع اکتشافی زیر را در نظر بگیرید:

$$h_1(A) = 1, \quad h_1(G) = 0.$$

این تابع یکنوا است، زیرا:

$$h_1(A) \leq c(A, G) + h_1(G).$$

یعنی:

$$1 \leq 1 + 0.$$

حال اگر عدد ثابت را برابر ۲ بگیریم، داریم:

$$h_2 = 2h_1.$$

پس:

$$h_2(A) = 2, \quad h_2(G) = 0.$$

برای یکنوا بودن باید داشته باشیم:

$$h_2(A) \leq c(A, G) + h_2(G).$$

یعنی:

$$2 \not\leq 1 + 0.$$

که این با یکنوا بودن h_2 در تناقض است.

(د) **درست** در جستجوی A^* در حالت درختی، اگر تابع اکتشافی h قابل قبول یا admissible باشد، یعنی هیچ وقت هزینه‌ی واقعی باقی مانده تا هدف را بیش از مقدار بهینه برآورد نکند:

$$h(n) \leq h^*(n),$$

آنگاه A^* درختی جواب بهینه را پیدا می‌کند.

تابع ارزیابی در A^* برابر است با:

$$f(n) = g(n) + h(n),$$

که در آن $g(n)$ هزینه‌ی مسیر طی شده از شروع تا n ، و $h(n)$ تخمین هزینه‌ی باقی مانده از n تا هدف است.

چون h قابل قبول است، مقدار $f(n)$ نیز هزینه‌ی کل مسیر بهینه‌ای را که از n عبور کند بیش برآورد نمی‌کند. بنابراین تا وقتی یک مسیر بهتر از جواب فعلی ممکن باشد، A^* آن را کاملاً نادیده نمی‌گیرد. به عبارتی جستجوی درختی (شرایط استاندارد)، قابل قبول بودن برای تضمین بهینگی کافی است. اما در جستجوی گراف، معمولاً برای تضمین بهینگی بدون بازگشایی گره‌ها، شرط قوی‌تری یعنی یکنوا بودن لازم است.

(ه) **نادرست.** در الگوریتم Simulated Annealing، در هر مرحله معمولاً یک همسایه انتخاب می‌شود. اگر آن همسایه بهتر باشد، پذیرفته می‌شود. اگر بدتر باشد، ممکن است با احتمالی وابسته به دما پذیرفته شود.

برای مسئله‌ی کمینه سازی، احتمال پذیرش حرکت بدتر معمولاً چیزی از جنس زیر است:

$$P = e^{-\Delta E/T},$$

که در آن ΔE میزان بدتر شدن جواب و T دما است. وقتی دما کاهش پیدا می‌کند و به صفر نزدیک می‌شود، احتمال پذیرش حرکت های بدتر تقریباً صفر می‌شود. بنابراین الگوریتم بیشتر شبیه یک نوع hill climbing تصادفی رفتار می‌کند، نه Local Beam Search با $k = 1$.

در Local Beam Search با $k = 1$ ، فقط یک حالت نگه داشته می‌شود و معمولاً بهترین جانشین انتخاب می‌شود؛ این رفتار بیشتر به steepest-ascent hill climbing شباهت دارد. اما Simulated Annealing حتی در دمای پایین نیز ساختار متفاوتی دارد، زیرا معمولاً یک همسایه را به صورت تصادفی بررسی می‌کند و سیاست پذیرش آن وابسته به دما است.

پس کاهش دما باعث می‌شود Simulated Annealing کمتر حرکت بدتر را قبول کند، اما آن را به Local Beam Search با $k = 1$ تبدیل نمی‌کند.

(و) **نادرست.** الگوریتم Iterative Deepening Search چندین بار جستجوی عمقی محدود انجام می‌دهد. ابتدا تا عمق ۰، سپس تا عمق ۱، سپس تا عمق ۲ و همین‌طور تا رسیدن به عمق جواب ادامه می‌دهیم.

در مقابل، BFS هر گره را در سطح خودش فقط یک بار تولید یا گسترش می‌دهد. اگر عمق جواب را d و عامل انشعاب را b بگیریم، در بدترین حالت، BFS تقریباً گره‌های زیر را تا عمق d بررسی می‌کند:

$$1 + b + b^2 + \dots + b^d.$$

اما IDS گره‌های نزدیک ریشه را چندین بار باز می‌کند. تعداد گسترش گره ها در IDS چیزی از فرمول زیر در بدترین حالت است:

$$(d + 1) \cdot 1 + d \cdot b + (d - 1) \cdot b^2 + \dots + 1 \cdot b^d.$$

پس تعداد گسترش‌های IDS دقیقاً برابر با BFS نیست. معمولاً بیشتر است، چون سطوح کم عمق بارها تکرار می‌شوند.

اما از نظر مرتبه‌ی رشد، اگر $b > 1$ باشد، هر دو از مرتبه‌ی:

$$O(b^d)$$

هستند.

(ز) **درست.** الگوریتم Uniform Cost Search همیشه گره ای را گسترش می‌دهد که کمترین هزینه‌ی مسیر از شروع تا آن را دارد.

اگر تمام هزینه‌ی یال‌ها حداقل یک مقدار مثبت $\epsilon > 0$ باشند، آنگاه تعداد مسیرهایی که هزینه‌ی کمتر از هزینه‌ی جواب بهینه دارند محدود می‌شود و UCS به جواب می‌رسد.

اما اگر یال‌هایی با هزینه‌ی صفر وجود داشته باشند، ممکن است تعداد بی‌نهایتی مسیر یا حالت با هزینه‌ی صفر یا با هزینه‌ی کمتر از هزینه‌ی جواب تولید شود. در این حالت UCS ممکن است هیچ‌وقت به سراغ جواب با هزینه‌ی مثبت نرود، حتی اگر چنین جوابی وجود داشته باشد. برای مثال، فرض کنید از گره شروع S دو مسیر داریم:

$$S \rightarrow G$$

با هزینه:

۱

و همچنین یک زنجیره ی بی نهایت:

$$S \rightarrow A_1 \rightarrow A_2 \rightarrow A_3 \rightarrow \dots$$

که هزینه ی همه ی یال های آن صفر است:

.

در این حالت، جواب G با هزینه ی متناهی ۱ وجود دارد. اما UCS همیشه گره هایی با هزینه ی صفر را نسبت به گره G با هزینه ی ۱ در اولویت قرار می دهد. چون زنجیره ی صفرهزینه بی نهایت است، ممکن است الگوریتم هیچ وقت به G نرسد.

پس وجود یال های صفرهزینه می تواند شرط خاتمه ی UCS را از بین ببرد.

(ح) درست. می دانیم اگر h یک نوا (consistent) باشد، برای هر یال (n, n') داریم:

$$h(n) \leq c(n, n') + h(n')$$

از آنجا که h_1 و h_2 یک نوا هستند:

$$h_1(n) \leq c(n, n') + h_1(n')$$

$$h_2(n) \leq c(n, n') + h_2(n')$$

حال اگر تعریف کنیم:

$$h(n) = \min(h_1(n), h_2(n))$$

داریم:

$$h(n) = \min(h_1(n), h_2(n)) \leq \min(c+h_1(n'), c+h_2(n')) = c+\min(h_1(n'), h_2(n')) = c+h(n')$$

پس شرط یک نوایی برقرار است.

(ط) نادرست. در Beam Search در هر مرحله تنها بهترین K حالت نگه داری می شوند. با وجود این که افزایش K تنوع جستجو را بیشتر می کند، اما با فرض محدود بودن زمان و حافظه:

- با بزرگ تر شدن K ، تعداد حالات تولیدی در هر مرحله افزایش می یابد.
- ممکن است عمق کمتری از فضا کاوش شود.
- ممکن است پیش از رسیدن به هدف، محدودیت زمان/حافظه فعال شود.

بنابراین افزایش K لزوماً باعث افزایش احتمال موفقیت نمی شود.

(ی) نادرست. Hill Climbing در هر مرحله بهترین همسایه را انتخاب می کند. اگر از S به G رسیده باشد، یعنی یک مسیر greedy موفق وجود داشته است.

اما در Beam Search در هر مرحله بهترین K حالت از میان تمام جانشین های جمعیت فعلی انتخاب می شوند. ممکن است حالتی که روی مسیر صحیح قرار دارد، در بین K حالت برتر قرار نگیرد و حذف شود. در این صورت مسیر صحیح، حتی اگر Hill Climbing آن را انتخاب می کرد، از بین می رود.

بنابراین موفقیت Hill Climbing لزوماً موفقیت Beam Search را تضمین نمی کند.

۲. (۱۰ نمره) درخت کاملی با ضریب انشعاب b و عمق d را در نظر بگیرید. هدف در عمق m قرار دارد.

- (آ) در بهترین و بدترین حالت، تعداد گره‌های بسط داده شده توسط DFS و BFS را محاسبه کنید.
- (ب) سه حالت مختلف را برای گره هدف (که در عمق m قرار دارد) در نظر بگیرید: هدف چپ‌ترین گره در این عمق باشد، هدف راست‌ترین گره در این عمق باشد و یا هدف به احتمال برابر در یکی از گره‌های موجود در این عمق قرار داشته باشد. در هرکدام از این سه حالت عملکرد الگوریتم‌های BFS و DFS را مقایسه کرده و بگویید استفاده از کدام یک بهتر است.
- (ج) برای چه مقادیری از m و d ، استفاده از BFS منطقی‌تر است؟

حل. فرض: درخت کامل با ضریب انشعاب b و عمق d ، و هدف در عمق m قرار دارد.
مجموع گره‌های تا عمق k :

$$1 + b + b^2 + \dots + b^k = \frac{b^{k+1} - 1}{b - 1}$$

(آ) BFS: سطح به سطح پیش می‌رود.

بهترین حالت: هدف اولین گره در سطح m باشد. بنابراین BFS باید تمام سطوح قبل را کامل بسط دهد:

$$1 + b + b^2 + \dots + b^{m-1} = \frac{b^m - 1}{b - 1}$$

و سپس اولین گره سطح m را بررسی کند. بنابراین:

$$N_{BFS}^{best} = \frac{b^m - 1}{b - 1} + 1$$

بدترین حالت: هدف آخرین گره سطح m باشد. در این صورت کل سطح m نیز بررسی می‌شود:

$$N_{BFS}^{worst} = 1 + b + b^2 + \dots + b^m = \frac{b^{m+1} - 1}{b - 1}$$

DFS: ابتدا تا عمق ممکن پایین می‌رود.

بهترین حالت: مسیر هدف در اولین شاخه انتخابی باشد. آنگاه فقط مسیر ریشه تا هدف بررسی می‌شود:

$$N_{DFS}^{best} = m + 1$$

بدترین حالت: هدف در آخرین شاخه بررسی شود. آنگاه DFS تقریباً کل درخت را تا عمق d می‌گردد:

$$N_{DFS}^{worst} = \frac{b^{d+1} - 1}{b - 1}$$

که از مرتبه $O(b^d)$ است.

(ب) **حالت ۱: هدف چپ‌ترین گره سطح m باشد**

- DFS: تقریباً $m + 1$ گره بررسی می‌شود.
- BFS: از مرتبه $O(b^m)$ خواهد بود.

بنابراین DFS بهتر است.

حالت ۲: هدف راست‌ترین گره سطح m باشد

• DFS: از مرتبه $O(b^d)$ خواهد بود.

• BFS: از مرتبه $O(b^m)$ خواهد بود.

اگر $d \ll m$ باشد:

$$b^m \ll b^d$$

پس BFS بهتر است.

حالت ۳: هدف به طور یکنواخت تصادفی در سطح m باشد

• DFS: ممکن است چندین زیردرخت عمیق تا عمق d را بررسی کند، بنابراین مرتبه میانگین نزدیک به $O(b^d)$ است (خصوصاً اگر $d \ll m$ باشد).

• BFS: همیشه تا عمق m پیش می‌رود و به طور میانگین نصف گره‌های سطح m بررسی می‌شوند، پس مرتبه کلی $O(b^m)$ است.

در حالت میانگین، BFS معمولاً پایدارتر و بهتر است.

(ج) استفاده از BFS زمانی منطقی‌تر است که:

$$m \ll d$$

یعنی هدف کم‌عمق و درخت عمیق باشد. در این حالت:

$$BFS = O(b^m)$$

$$DFS = O(b^d)$$

اختلاف نمایی خواهد بود.

اما اگر $m \approx d$ باشد، هر دو الگوریتم تقریباً از مرتبه $O(b^d)$ هستند و تفاوت چشمگیری وجود ندارد.

۳. (۲۰ نمره) مسئله‌ی مرتب‌سازی n عدد صحیح متمایز را در نظر بگیرید ($n \geq 3$). فرض کنید در هر مرحله می‌توانیم دو عضو همسایه را با هم جابه‌جا کنیم. می‌خواهیم دنباله را به صورت صعودی مرتب کنیم.

(آ) فضای مسئله را به گونه‌ای مدل‌سازی کنید که حالت‌ها، کنش‌ها، حالت اولیه و حالت نهایی مسئله مشخص باشد.

(ب) دو تابع fitness مختلف ارائه کنید: تابع اول که نقطه بهینه محلی غیرسراسری داشته باشد و تابع دوم که دارای چنین نقطه‌ای نباشد.

(ج) با در نظر گرفتن تابع fitness دومی که در بخش قبل ارائه کردید، الگوریتم Hill Climbing را مرحله به مرحله تا اتمام الگوریتم روی دنباله‌ی (۸, ۹, ۲۴, ۱۵) اجرا کنید.

(د) با در نظر گرفتن تابع دوم و جدول دمای زیر، ۸ مرحله اول الگوریتم Simulated Annealing را روی دنباله‌ی بخش (ج) اجرا کنید:

t	۱	۲	۳	۴	۵	≥ 6
T	۵	۴	۳	۲	۱	۰

در اجرای این الگوریتم، هنگام نمونه‌برداری کردن همسایه‌های یک حالت، به ترتیب جابه‌جایی‌ها را از چپ به راست انجام دهید. این یعنی اولین همسایه نمونه‌برداری شده از یک حالت، با جابه‌جا شدن دو عضو اول دنباله آن حالت به دست می‌آید، و دومین همسایه نیز با جابه‌جا شدن اعضای دوم و سوم آن دنباله ساخته می‌شود. برای نمونه‌برداری از هر حالت، ابتدا الگوریتم را روی اولین همسایه آن (مطابق ترتیب تعریف شده) اعمال کنید، و اگر این همسایه به عنوان حالت بعدی انتخاب نشد به همسایه دوم و سوم رفته و اگر هیچ‌کدام انتخاب نشدند الگوریتم را به پایان برسانید. برای تولید اعداد تصادفی جهت انتخاب یا عدم انتخاب همسایه نیز می‌توانید از اعداد زیر استفاده کنید. (ممکن است تمامی اعداد استفاده نشوند.)

$$\{0.27, 0.66, 0.91, 0.13, 0.62, 0.36, 0.04\}$$

حل.

- (آ) • حالت‌ها: هر حالت جایگشتی از n عدد ورودی است.
 • کنش‌ها: جابه‌جا کردن دو عضو همسایه
 • حالت اولیه: دنباله‌ی ورودی داده شده
 • حالت نهایی: دنباله‌ی شامل اعداد ورودی که به صورت صعودی مرتب شده است.
- (ب) • تابع اول: به هر دنباله‌ای تعداد جفت‌عناصر همسایه‌ای در آن که عضو اول کمتر از عضو دوم باشد را نسبت می‌دهیم. به عنوان مثال، مقدار این تابع به ازای دنباله‌ی (۵, ۴, ۹, ۳, ۶) برابر با ۲ است، زیرا در جفت‌عناصر (۳, ۹) و (۴, ۵) عضو اول از عضو دوم کمتر است. این تابع بهینه‌ی محلی غیرسراسری دارد. به عنوان مثال، به ازای $n = 3$ دنباله‌ی (۴, ۱, ۷) را در نظر بگیرید. مقدار این تابع به ازای این دنباله برابر با ۱ است (که می‌دانیم بیشترین مقدار ممکن در فضای حالات نیست و دنباله‌ی مرتب‌شده خروجی ۲ می‌دهد). دو کنش برای این دنباله داریم؛ یا باید عضو اول و دوم را جابه‌جا کنیم یا عضو دوم و سوم. با انجام هر یک از این کنش‌ها به ترتیب به دنباله‌های (۴, ۱, ۷) و (۷, ۴, ۱) می‌رسیم که مقدار تابع به ازای آن‌ها به ترتیب ۱ و ۰ است. بنابراین این دنباله از همسایه‌های خود بدتر نیست، اما در کل فضای حالات بهترین fitness را ندارد؛ بنابراین یک بهینه‌ی محلی غیرسراسری است.
- تابع دوم: به هر دنباله‌ای تعداد جفت‌عناصری (نه لزوماً همسایه) در آن که عضو اول کمتر از عضو دوم باشد را نسبت می‌دهیم. به عنوان مثال، مقدار این تابع به ازای دنباله‌ی (۵, ۴, ۹, ۳, ۶) برابر با ۵ است، زیرا در جفت‌عناصر (۶, ۹)، (۳, ۹)، (۳, ۴)، (۳, ۵) و (۴, ۵) عضو اول از عضو دوم کمتر است. اثبات می‌کنیم که این تابع بهینه‌ی محلی غیرسراسری ندارد.
- لم ۱: با جابه‌جا کردن دو عضو همسایه، مقدار این تابع یا یک واحد زیاد و یا یک واحد کم می‌شود.
- اثبات لم ۱: فرض کنید در یک کنش، دو عضو در جایگاه i و $i + 1$ را جابه‌جا کنیم. در این صورت تمام جفت‌عناصری که شامل یکی از این دو عضو و عضوی دیگر در جایگاه‌های قبل از i یا بعد از $i + 1$ هستند، ترتیب نسبی یکسانی با حالت قبل از کنش خواهد داشت (چون با انجام کنش، فقط جای دو عضو متوالی i و $i + 1$ نسبت به هم عوض می‌شود). بنابراین سهم آن‌ها در مقدار تابع تغییر نمی‌کند. تنها جفتی که وضعیت آن عوض می‌شود همان جفت $(i, i + 1)$ است. اگر پیش از جابه‌جایی داشته باشیم $a_i < a_{i+1}$ ، آن‌گاه پس از جابه‌جایی مقدار تابع یک واحد کم می‌شود؛ در غیر این صورت مقدار تابع یک واحد افزایش می‌یابد.
- لم ۲: در هر دنباله‌ی نامرتب، حداقل یک وارونگی همسایگی (دو عنصر مجاور با ترتیب نزولی) وجود دارد.
- اثبات لم ۲: اگر دنباله مرتب نباشد، آن‌گاه امکان ندارد برای هر i رابطه‌ی $a_i < a_{i+1}$ برقرار باشد، زیرا در این صورت کل دنباله صعودی می‌شود که مرتب‌شده است. بنابراین حداقل یک اندیس i وجود دارد که $a_i > a_{i+1}$ ، که همان وارونگی همسایگی است.

اکنون نشان می‌دهیم این تابع هیچ بهینه‌ی محلی غیرسراسری ندارد. فرض کنید دنباله‌ای یک بهینه‌ی محلی باشد. اگر این دنباله مرتب نباشد، طبق لم ۲ دارای یک وارونگی همسایگی است. با جابه‌جا کردن این دو عنصر، طبق لم ۱ مقدار تابع یک واحد افزایش می‌یابد. بنابراین این دنباله همسایه‌ای با fitness اکیداً بیشتر دارد و نمی‌تواند یک بهینه‌ی محلی باشد. این تناقض نشان می‌دهد که هر بهینه‌ی محلی باید حتماً دنباله‌ای مرتب (صعودی) باشد. از آنجا که تنها دنباله‌ی مرتب همان بهینه‌ی سراسری (با مقدار) است، نتیجه می‌گیریم که این تابع هیچ بهینه‌ی محلی غیرسراسری ندارد. (ج) مرحله اول:

$$\begin{aligned} \text{current} &\rightarrow (15, 24, 9, 8) \rightarrow f = 1 \\ \text{neighbors} &\rightarrow (24, 15, 9, 8) \rightarrow f = 0 \\ &\rightarrow (15, 9, 24, 8) \rightarrow f = 2 \\ &\rightarrow (15, 24, 8, 9) \rightarrow f = 2 \end{aligned}$$

در هر مرحله، از بین بهترین همسایه‌ها، یکی به دلخواه انتخاب می‌شود (در این مرحله، یکی از همسایه‌های دوم و سوم). فرض می‌کنیم همسایه‌ی دوم انتخاب شود. مرحله دوم:

$$\begin{aligned} \text{current} &\rightarrow (15, 9, 24, 8) \rightarrow f = 2 \\ \text{neighbors} &\rightarrow (9, 15, 24, 8) \rightarrow f = 3 \\ &\rightarrow (15, 24, 9, 8) \rightarrow f = 1 \\ &\rightarrow (15, 9, 8, 24) \rightarrow f = 3 \end{aligned}$$

یکی از همسایه‌های اول و سوم به دلخواه انتخاب می‌شود. فرض می‌کنیم همسایه‌ی اول انتخاب شود. مرحله سوم:

$$\begin{aligned} \text{current} &\rightarrow (9, 15, 24, 8) \rightarrow f = 3 \\ \text{neighbors} &\rightarrow (15, 9, 24, 8) \rightarrow f = 2 \\ &\rightarrow (9, 24, 15, 8) \rightarrow f = 2 \\ &\rightarrow (9, 15, 8, 24) \rightarrow f = 4 \end{aligned}$$

همسایه‌ی سوم انتخاب می‌شود. مرحله چهارم:

$$\begin{aligned} \text{current} &\rightarrow (9, 15, 8, 24) \rightarrow f = 4 \\ \text{neighbors} &\rightarrow (15, 9, 8, 24) \rightarrow f = 3 \\ &\rightarrow (9, 8, 15, 24) \rightarrow f = 5 \\ &\rightarrow (9, 15, 24, 8) \rightarrow f = 3 \end{aligned}$$

همسایه‌ی دوم انتخاب می‌شود. مرحله پنجم:

$$\begin{aligned} \text{current} &\rightarrow (9, 8, 15, 24) \rightarrow f = 5 \\ \text{neighbors} &\rightarrow (8, 9, 15, 24) \rightarrow f = 6 \\ &\rightarrow (9, 15, 8, 24) \rightarrow f = 4 \\ &\rightarrow (9, 8, 24, 15) \rightarrow f = 4 \end{aligned}$$

همسایه‌ی اول انتخاب می‌شود. مرحله ششم:

$$\begin{aligned} \text{current} &\rightarrow (8, 9, 15, 24) \rightarrow f = 6 \\ \text{neighbors} &\rightarrow (9, 8, 15, 24) \rightarrow f = 5 \\ &\rightarrow (8, 15, 9, 24) \rightarrow f = 5 \\ &\rightarrow (8, 9, 24, 15) \rightarrow f = 5 \end{aligned}$$

هیچ یک از همسایه‌ها بهتر از حالت فعلی نیست؛ بنابراین اجرای الگوریتم تمام می‌شود. دنباله‌ی نهایی به دست آمده $(۸, ۹, ۱۵, ۲۴)$ است که همان دنباله‌ی مرتب شده است. توجه کنید که چون طبق بخش ب، تابع تعریف شده بهینه‌ی محلی غیرسراسری ندارد، الگوریتم Hill Climbing حتما در نهایت به دنباله‌ی مرتب‌شده می‌رسد.

(د) گام ۱ ($t = ۱, T = ۵$): حالت فعلی:

$$s = (۱۵, ۲۴, ۹, ۸), f(s) = ۱$$

• بررسی همسایه ۱ (جابه‌جایی ۱-۲):

$$s' = (۲۴, ۱۵, ۹, ۸), f(s') = ۰ \implies \Delta f = -۱$$

• احتمال پذیرش: $P = e^{-1/5} \approx ۰/۸۱۸۷$

• عدد تصادفی: $۰/۸۱۸۷ \leq ۰/۲۷$ پس پذیرفته شد.

حالت جدید: $(۲۴, ۱۵, ۹, ۸)$

گام ۲ ($t = ۲, T = ۴$): حالت فعلی:

$$s = (۲۴, ۱۵, ۹, ۸), f(s) = ۰$$

• بررسی همسایه ۱ (جابه‌جایی ۱-۲):

$$s' = (۱۵, ۲۴, ۹, ۸), f(s') = ۱ \implies \Delta f = ۱ \geq ۰$$

• به‌طور خودکار پذیرفته شد.

حالت جدید: $(۱۵, ۲۴, ۹, ۸)$

گام ۳ ($t = ۳, T = ۳$): حالت فعلی:

$$s = (۱۵, ۲۴, ۹, ۸), f(s) = ۱$$

• بررسی همسایه ۱ (جابه‌جایی ۱-۲):

$$s' = (۲۴, ۱۵, ۹, ۸), f(s') = ۰ \implies \Delta f = -۱$$

• احتمال پذیرش: $P = e^{-1/3} \approx ۰/۷۱۶۵$

• عدد تصادفی: $۰/۷۱۶۵ \leq ۰/۶۶$ پس پذیرفته شد.

حالت جدید: $(۲۴, ۱۵, ۹, ۸)$

گام ۴ ($t = ۴, T = ۲$): حالت فعلی:

$$s = (۲۴, ۱۵, ۹, ۸), f(s) = ۰$$

• بررسی همسایه ۱ (جابه‌جایی ۱-۲):

$$s' = (۱۵, ۲۴, ۹, ۸), f(s') = ۱ \implies \Delta f = ۱ \geq ۰$$

• به‌طور خودکار پذیرفته شد.

حالت جدید: $(۱۵, ۲۴, ۹, ۸)$

گام ۵ ($t = ۵, T = ۱$): حالت فعلی:

$$s = (۱۵, ۲۴, ۹, ۸), f(s) = ۱$$

- بررسی همسایه ۱ (جابه‌جایی ۱-۲):

$$s' = (24, 15, 9, 8), f(s') = 0 \implies \Delta f = -1$$

- احتمال پذیرش: $P = e^{-1/1} \approx 0.3678$
- عدد تصادفی: $0.91 > 0.3678$ پس رد شد.
- حالت فعلی باقی می‌ماند: $(15, 24, 9, 8)$
- گام ۶ ($t = 6, T = 0$): حالت فعلی:

$$s = (15, 24, 9, 8), f(s) = 1$$

- بررسی همسایه ۲ (جابه‌جایی ۲-۳):

$$s' = (15, 9, 24, 8), f(s') = 2 \implies \Delta f = 1 \geq 0$$

- به‌طور خودکار پذیرفته شد.
- حالت جدید: $(15, 9, 24, 8)$
- گام ۷ ($t = 7, T = 0$): حالت فعلی:

$$s = (15, 9, 24, 8), f(s) = 2$$

- بررسی همسایه ۱ (جابه‌جایی ۱-۲):

$$s' = (9, 15, 24, 8), f(s') = 3 \implies \Delta f = 1 \geq 0$$

- به‌طور خودکار پذیرفته شد.
- حالت جدید: $(9, 15, 24, 8)$
- گام ۸ ($t = 8, T = 0$): حالت فعلی:

$$s = (9, 15, 24, 8), f(s) = 3$$

- بررسی همسایه ۱ (جابه‌جایی ۱-۲):

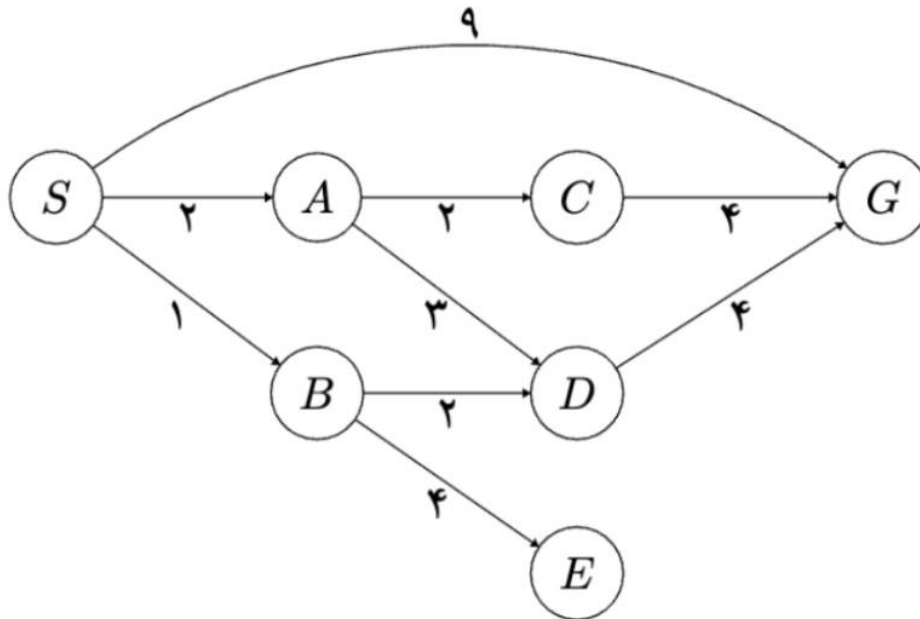
$$s' = (15, 9, 24, 8), f(s') = 2 \implies \Delta f = -1 < 0$$

- برای $T = 0$ ، هر حالتی با $\Delta f < 0$ رد می‌شود ($P = 0$).
- حالت فعلی باقی می‌ماند: $(9, 15, 24, 8)$

۴. (۲۰ نمره) گراف جستجوی زیر را در نظر بگیرید. S راس شروع و G هدف است. هزینه هر یال بر روی آن نوشته شده است و یال‌ها دو طرفه اند. برای هر یک از الگوریتم‌های زیر مشخص کنید کدام راس در هر مرحله باز می‌شود و مقدار هر گره چگونه بروزرسانی می‌شوند. مسیری که هر استراتژی خروجی می‌دهد را مشخص کنید (اولویت باز کردن رئوس طبق ترتیب حروف انگلیسی است).

- جستجوی BFS
- جستجوی DFS
- جستجوی UCS
- جستجوی Greedy

- جستجوی درختی A^*
- جستجوی گراف A^*



هیوریستیک

G	E	D	C	B	A	S
۰	۱۰	۱	۴	۶	۰	۶

حل.

(آ) جستجوی BFS:

- مقداردهی اولیه: صف $[(S)]$
- گام ۱: بسط (S) . همسایه‌ها به ترتیب الفبا اضافه می‌شوند: $(S \rightarrow A), (S \rightarrow B), (S \rightarrow G)$.
صف $[(S \rightarrow A), (S \rightarrow B), (S \rightarrow G)]$
- گام ۲: بسط $(S \rightarrow A)$. همسایه‌ها اضافه می‌شوند: $(S \rightarrow A \rightarrow C), (S \rightarrow A \rightarrow D)$.
صف $[(S \rightarrow B), (S \rightarrow G), (S \rightarrow A \rightarrow C), (S \rightarrow A \rightarrow D)]$
- گام ۳: بسط $(S \rightarrow B)$. همسایه‌ها اضافه می‌شوند: $(S \rightarrow B \rightarrow D), (S \rightarrow B \rightarrow E)$.
صف $[(S \rightarrow G), (S \rightarrow A \rightarrow C), (S \rightarrow A \rightarrow D), (S \rightarrow B \rightarrow D), (S \rightarrow B \rightarrow E)]$
- گام ۴: بسط $(S \rightarrow G)$. هدف پیدا شد.

مسیر نهایی یافت شده: $S \rightarrow G$
هزینه مسیر: ۹

(ب) جستجوی DFS:

برای حفظ اولویت الفبایی در طول بسط، همسایه‌ها به ترتیب معکوس الفبایی وارد پشته می‌شوند.

- مقداردهی اولیه: پشته = $[(S)]$
- گام ۱: بسط (S) . وارد کردن $(S \rightarrow A), (S \rightarrow B), (S \rightarrow G)$.
پشته = $[(S \rightarrow A), (S \rightarrow B), (S \rightarrow G)]$ (رأس پشته $(S \rightarrow A)$ است)
- گام ۲: بسط $(S \rightarrow A)$. وارد کردن $(S \rightarrow A \rightarrow D), (S \rightarrow A \rightarrow C)$.
پشته = $[(S \rightarrow A \rightarrow C), (S \rightarrow A \rightarrow D), (S \rightarrow B), (S \rightarrow G)]$ (رأس پشته $(S \rightarrow A \rightarrow C)$ است)
- گام ۳: بسط $(S \rightarrow A \rightarrow C)$. وارد کردن $(S \rightarrow A \rightarrow C \rightarrow G)$.
پشته = $[(S \rightarrow A \rightarrow C \rightarrow G), (S \rightarrow A \rightarrow D), (S \rightarrow B), (S \rightarrow G)]$ (رأس پشته $(S \rightarrow A \rightarrow C \rightarrow G)$ است)
- گام ۴: بسط $(S \rightarrow A \rightarrow C \rightarrow G)$. هدف پیدا شد.

مسیر نهایی یافت شده: $S \rightarrow A \rightarrow C \rightarrow G$

هزینه مسیر: ۸

(ج) جستجوی UCS:

- مقداردهی اولیه: صف = $[(S), g = ۰]$
- گام ۱: بسط (S) ($g = ۰$). کشف $(S \rightarrow A)(g = ۲), (S \rightarrow B)(g = ۱), (S \rightarrow G)(g = ۹)$.
صف = $[(S \rightarrow B), ۱), ((S \rightarrow A), ۲), ((S \rightarrow G), ۹)]$
- گام ۲: بسط $(S \rightarrow B)$ ($g = ۱$). کشف $(S \rightarrow B \rightarrow D)(g = ۱ + ۲ = ۳), (S \rightarrow B \rightarrow E)(g = ۱ + ۴ = ۵)$.
صف = $[(S \rightarrow A), ۲), ((S \rightarrow B \rightarrow D), ۳), ((S \rightarrow B \rightarrow E), ۵), ((S \rightarrow G), ۹)]$
- گام ۳: بسط $(S \rightarrow A)$ ($g = ۲$). کشف $(S \rightarrow A \rightarrow C)(g = ۲ + ۲ = ۴)$ و $(S \rightarrow A \rightarrow D)(g = ۲ + ۳ = ۵)$.
صف = $[(S \rightarrow B \rightarrow D), ۳), ((S \rightarrow A \rightarrow C), ۴), ((S \rightarrow A \rightarrow D), ۵), ((S \rightarrow B \rightarrow E), ۵), ((S \rightarrow G), ۹)]$
- گام ۴: بسط $(S \rightarrow B \rightarrow D)$ ($g = ۳$). کشف مسیر به G از طریق D به صورت $(S \rightarrow B \rightarrow D \rightarrow G)$ با هزینه $g = ۳ + ۴ = ۷$.
صف = $[(S \rightarrow A \rightarrow C), ۴), ((S \rightarrow A \rightarrow D), ۵), ((S \rightarrow B \rightarrow E), ۵), ((S \rightarrow B \rightarrow D \rightarrow G), ۷), ((S \rightarrow G), ۹)]$
- گام ۵: بسط $(S \rightarrow A \rightarrow C)$ ($g = ۴$). کشف مسیر به G از طریق C به صورت $(S \rightarrow A \rightarrow C \rightarrow G)$ با هزینه $g = ۴ + ۴ = ۸$.
صف = $[(S \rightarrow A \rightarrow D), ۵), ((S \rightarrow B \rightarrow E), ۵), ((S \rightarrow B \rightarrow D \rightarrow G), ۷), ((S \rightarrow A \rightarrow C \rightarrow G), ۸), ((S \rightarrow G), ۹)]$
- گام ۶: بسط $(S \rightarrow A \rightarrow D)$ ($g = ۵$). کشف مسیر به G از طریق D به صورت $(S \rightarrow A \rightarrow D \rightarrow G)$ با هزینه $g = ۵ + ۴ = ۹$.
صف = $[(S \rightarrow B \rightarrow E), ۵), ((S \rightarrow B \rightarrow D \rightarrow G), ۷), ((S \rightarrow A \rightarrow C \rightarrow G), ۸), ((S \rightarrow A \rightarrow D \rightarrow G), ۹), ((S \rightarrow G), ۹)]$
- گام ۷: بسط $(S \rightarrow B \rightarrow E)$ ($g = ۵$). مسیر جدیدی پیدا نمی شود.
صف = $[(S \rightarrow B \rightarrow D \rightarrow G), ۷), ((S \rightarrow A \rightarrow C \rightarrow G), ۸), ((S \rightarrow A \rightarrow D \rightarrow G), ۹), ((S \rightarrow G), ۹)]$
- گام ۸: بسط $(S \rightarrow B \rightarrow D \rightarrow G)$ ($g = ۷$). هدف پیدا شد.

مسیر نهایی یافت شده: $S \rightarrow B \rightarrow D \rightarrow G$

هزینه مسیر: ۷

(د) جستجوی Greedy:

- مقداردهی اولیه: صف = $[(S), h = ۶]$

- **گام ۱:** بسط (S) . کشف $(S \rightarrow A)(h = \circ), (S \rightarrow B)(h = \epsilon), (S \rightarrow G)(h = \circ)$. گره‌های هم‌امتیاز A و G به ترتیب الفبا مرتب می‌شوند.
صف $[(S \rightarrow A), \circ], [(S \rightarrow G), \circ], [(S \rightarrow B), \epsilon]$
- **گام ۲:** بسط $(S \rightarrow A)$ ($h = \circ$). کشف $(S \rightarrow A \rightarrow C)(h = \epsilon)$ و $(S \rightarrow A \rightarrow D)(h = \circ)$.
صف $[(S \rightarrow G), \circ], [(S \rightarrow A \rightarrow D), \circ], [(S \rightarrow A \rightarrow C), \epsilon], [(S \rightarrow B), \epsilon]$
- **گام ۳:** بسط $(S \rightarrow G)$ ($h = \circ$). هدف پیدا شد.

مسیر نهایی یافت‌شده: $S \rightarrow G$

هزینه مسیر: ۹

(ه) جستجوی درختی A^* :

- **مقداردهی اولیه:** صف $[(S), g = \circ, h = \epsilon, f = \epsilon]$
- **گام ۱:** بسط (S) .
- مسیر $(S \rightarrow A): g = 2, h = \circ \Rightarrow f = 2$
- مسیر $(S \rightarrow B): g = 1, h = \epsilon \Rightarrow f = 7$
- مسیر $(S \rightarrow G): g = 9, h = \circ \Rightarrow f = 9$
صف $[(S \rightarrow A), 2], [(S \rightarrow B), 7], [(S \rightarrow G), 9]$
- **گام ۲:** بسط $(S \rightarrow A)$ ($f = 2$).
- مسیر $(S \rightarrow A \rightarrow C): g = 4, h = \epsilon \Rightarrow f = 8$
- مسیر $(S \rightarrow A \rightarrow D): g = 5, h = \circ \Rightarrow f = 6$
صف $[(S \rightarrow A \rightarrow D), 6], [(S \rightarrow B), 7], [(S \rightarrow A \rightarrow C), 8], [(S \rightarrow G), 9]$
- **گام ۳:** بسط $(S \rightarrow A \rightarrow D)$ ($f = 6$).
- مسیر $(S \rightarrow A \rightarrow D \rightarrow G): g = 9, h = \circ \Rightarrow f = 9$ (تمامی مسیرهای متمایز در صف نگهداری می‌شوند).
صف $[(S \rightarrow B), 7], [(S \rightarrow A \rightarrow C), 8], [(S \rightarrow G), 9], [(S \rightarrow A \rightarrow D \rightarrow G), 9]$
- **گام ۴:** بسط $(S \rightarrow B)$ ($f = 7$).
- مسیر $(S \rightarrow B \rightarrow D): g = 3, h = \circ \Rightarrow f = 4$
- مسیر $(S \rightarrow B \rightarrow E): g = 5, h = \circ \Rightarrow f = 15$
صف $[(S \rightarrow B \rightarrow D), 4], [(S \rightarrow A \rightarrow C), 8], [(S \rightarrow G), 9], [(S \rightarrow A \rightarrow D \rightarrow G), 9], [(S \rightarrow B \rightarrow E), 15]$
- **گام ۵:** بسط $(S \rightarrow B \rightarrow D)$ ($f = 4$).
- مسیر $(S \rightarrow B \rightarrow D \rightarrow G): g = 7, h = \circ \Rightarrow f = 7$
صف $[(S \rightarrow B \rightarrow D \rightarrow G), 7], [(S \rightarrow A \rightarrow C), 8], [(S \rightarrow G), 9], [(S \rightarrow A \rightarrow D \rightarrow G), 9], [(S \rightarrow B \rightarrow E), 15]$
- **گام ۶:** بسط $(S \rightarrow B \rightarrow D \rightarrow G)$ ($f = 7$). هدف پیدا شد.

مسیر نهایی یافت‌شده: $S \rightarrow B \rightarrow D \rightarrow G$

هزینه مسیر: ۷

(و) جستجوی گراف A^* :

همراه با مجموعه کاوش‌شده (Explored Set) برای جلوگیری از بسط مجدد حالت‌ها.

- **مقداردهی اولیه:** صف $[(S), \epsilon]$ ، مجموعه کاوش‌شده $\emptyset$
- **گام ۱:** بسط (S) ($f = \epsilon$). مجموعه کاوش‌شده $\{S\}$.
صف $[(S \rightarrow A), 2], [(S \rightarrow B), 7], [(S \rightarrow G), 9]$

- **گام ۲:** بسط $(S \rightarrow A)$ ($f = ۲$). مجموعه کاوش شده $\{S, A\}$.
صف $= [((S \rightarrow A \rightarrow D), ۶), ((S \rightarrow B), ۷), ((S \rightarrow A \rightarrow C), ۸), ((S \rightarrow G), ۹)]$
- **گام ۳:** بسط $(S \rightarrow A \rightarrow D)$ ($f = ۶$). مجموعه کاوش شده $\{S, A, D\}$.
صف $= [((S \rightarrow B), ۷), ((S \rightarrow A \rightarrow C), ۸), ((S \rightarrow G), ۹), ((S \rightarrow A \rightarrow D \rightarrow G), ۹)]$
- **گام ۴:** بسط $(S \rightarrow B)$ ($f = ۷$). مجموعه کاوش شده $\{S, A, D, B\}$.
صف $= [((S \rightarrow B \rightarrow D), ۴), ((S \rightarrow A \rightarrow C), ۸), ((S \rightarrow G), ۹), ((S \rightarrow A \rightarrow D \rightarrow G), ۹), ((S \rightarrow B \rightarrow E), ۱۵)]$
- **گام ۵:** انتخاب $(S \rightarrow B \rightarrow D)$ ($f = ۴$). چون گره پایانی مسیر (D) قبلاً بسط داده شده و در مجموعه کاوش شده قرار دارد، این مسیر نادیده گرفته شده و بسط داده نمی شود (به دلیل ناسازگار بودن هیوریستیک، مسیر بهینه در اینجا مسدود می شود).
صف $= [((S \rightarrow A \rightarrow C), ۸), ((S \rightarrow G), ۹), ((S \rightarrow A \rightarrow D \rightarrow G), ۹), ((S \rightarrow B \rightarrow E), ۱۵)]$
- **گام ۶:** بسط $(S \rightarrow A \rightarrow C)$ ($f = ۸$). مجموعه کاوش شده $\{S, A, D, B, C\}$. کشف یک مسیر جدید به G به صورت $(S \rightarrow A \rightarrow C \rightarrow G)$ با هزینه $f = ۸$. (تمامی مسیرهای متمایز در صف نگهداری می شوند و جایگزینی صورت نمی گیرد).
صف $= [((S \rightarrow A \rightarrow C \rightarrow G), ۸), ((S \rightarrow G), ۹), ((S \rightarrow A \rightarrow D \rightarrow G), ۹), ((S \rightarrow B \rightarrow E), ۱۵)]$
- **گام ۷:** بسط $(S \rightarrow A \rightarrow C \rightarrow G)$ ($f = ۸$). هدف پیدا شد.
مسیر نهایی یافت شده: $S \rightarrow A \rightarrow C \rightarrow G$
هزینه مسیر: ۸

۵. (۱۵ نمره) یک الگوریتم ژنتیک را در نظر بگیرید که هر کروموزوم به صورت زیر نمایش داده می شود:

$$x = ABCDEF$$

هر ژن مقداری بین ۰ تا ۹ می گیرد. تابع fitness به صورت زیر تعریف شده است:

$$f(x) = ۲A + B + ۳C - D + ۲E + F$$

جمعیت اولیه:

$$x_1 = ۵۲۳۶۴۱, \quad x_2 = ۸۱۷۲۰۳, \quad x_3 = ۴۶۹۱۲۵, \quad x_4 = ۹۰۲۵۷۸$$

(آ) fitness هر کروموزوم را محاسبه کرده و بهترین آن را مشخص کنید.

(ب) تقاطع را به صورت زیر اعمال کنید:

- روی دو کروموزوم با بالاترین fitness، عملگر تقاطع تک نقطه ای (بعد از ژن سوم) اعمال کنید.
- روی دومین و سومین کروموزوم برتر، تقاطع دوماحوره انجام دهید به گونه ای که ژن های CD از یکی از والدین و بقیه ژن ها از والد دیگر بیایند.
- (ج) فرض کنید عملگر جهش همیشه و با احتمال یک روی تمامی جمعیت اعمال می شود، و طریقه اعمال آن با دو مقدار تصادفی x و y به صورت زیر است:
 - ژن در موقعیت x انتخاب می شود
 - مقدار y به صورت پیمانه ۱۰ به آن اضافه می شود

با مقادیر تصادفی $(x = ۳, y = ۴), (x = ۷, y = ۲), (x = ۱, y = ۵), (x = ۶, y = ۰)$ جهش را به ترتیب روی فرزندان اعمال کنید.
(د) میانگین fitness جمعیت جدید را با جمعیت اولیه مقایسه کنید.
حل.

(آ) ابتدا مقدار fitness هر کروموزوم را محاسبه می‌کنیم:

$$\begin{aligned} f(x_1) &= f(۵۲۳۶۴۱) = ۲(۵) + ۲ + ۳(۳) - ۶ + ۲(۴) + ۱ = ۲۴, \\ f(x_2) &= f(۸۱۷۲۰۳) = ۲(۸) + ۱ + ۳(۷) - ۲ + ۲(۰) + ۳ = ۳۹, \\ f(x_3) &= f(۴۶۹۱۲۵) = ۲(۴) + ۶ + ۳(۹) - ۱ + ۲(۲) + ۵ = ۴۹, \\ f(x_4) &= f(۹۰۲۵۷۸) = ۲(۹) + ۰ + ۳(۲) - ۵ + ۲(۷) + ۸ = ۴۱. \end{aligned}$$

بنابراین ترتیب کروموزوم‌ها از بهترین به ضعیف‌ترین برابر است با

$$x_3, x_4, x_2, x_1$$

و بهترین کروموزوم اولیه

$$x_3 = ۴۶۹۱۲۵$$

با مقدار fitness برابر با ۴۹ است.

(ب) تقاطع تک‌نقطه‌ای روی دو کروموزوم برتر، یعنی x_3 و x_4 ، بعد از ژن سوم انجام می‌شود:

$$x_3 = ۴۶۹|۱۲۵, \quad x_4 = ۹۰۲|۵۷۸.$$

پس دو فرزند حاصل از این تقاطع برابرند با

$$c_1 = ۴۶۹۵۷۸, \quad c_2 = ۹۰۲۱۲۵.$$

برای تقاطع دومحوره روی دومین و سومین کروموزوم برتر، یعنی x_2 و x_4 ، ژن‌های CD را از یکی از والدین و سایر ژن‌ها را از والد دیگر می‌گیریم:

$$x_4 = ۹۰۲۵۷۸, \quad x_2 = ۸۱۷۲۰۳.$$

بنابراین دو فرزند دیگر برابرند با

$$c_3 = ۹۰۷۲۷۸, \quad c_4 = ۸۱۲۵۰۳.$$

در نتیجه جمعیت فرزندان قبل از جهش به صورت زیر است:

$$۴۶۹۵۷۸, \quad ۹۰۲۱۲۵, \quad ۹۰۷۲۷۸, \quad ۸۱۲۵۰۳.$$

(ج) اکنون جهش‌ها را به ترتیب روی فرزندان اعمال می‌کنیم.
برای فرزند اول:

$$۴۶۹۵۷۸ \xrightarrow{(x=۳,y=۴)} ۴۶۳۵۷۸$$

زیرا ژن سوم برابر ۹ است و

$$(۹ + ۴) \bmod ۱۰ = ۳.$$

برای فرزند دوم:

$$۹۰۲۱۲۵ \xrightarrow{(x=۴,y=۲)} ۹۰۲۳۲۵$$

زیرا ژن چهارم برابر ۱ است و

$$(1 + 2) \bmod 10 = 3.$$

برای فرزند سوم:

$$907278 \xrightarrow{(x=1, y=5)} 407278$$

زیرا ژن اول برابر ۹ است و

$$(9 + 5) \bmod 10 = 4.$$

برای فرزند چهارم:

$$812503 \xrightarrow{(x=6, y=0)} 812503$$

زیرا مقدار اضافه شده صفر است و ژن ششم تغییری نمی‌کند.

پس جمعیت جدید بعد از جهش برابر است با

$$463578, \quad 902325, \quad 407278, \quad 812503.$$

(د) مقدار fitness جمعیت جدید را محاسبه می‌کنیم:

$$f(463578) = 2(4) + 6 + 3(3) - 5 + 2(7) + 8 = 40,$$

$$f(902325) = 2(9) + 0 + 3(2) - 3 + 2(2) + 5 = 30,$$

$$f(407278) = 2(4) + 0 + 3(7) - 2 + 2(7) + 8 = 49,$$

$$f(812503) = 2(8) + 1 + 3(2) - 5 + 2(0) + 3 = 21.$$

میانگین fitness جمعیت اولیه برابر است با

$$\frac{24 + 39 + 49 + 41}{4} = \frac{153}{4} = 38.25.$$

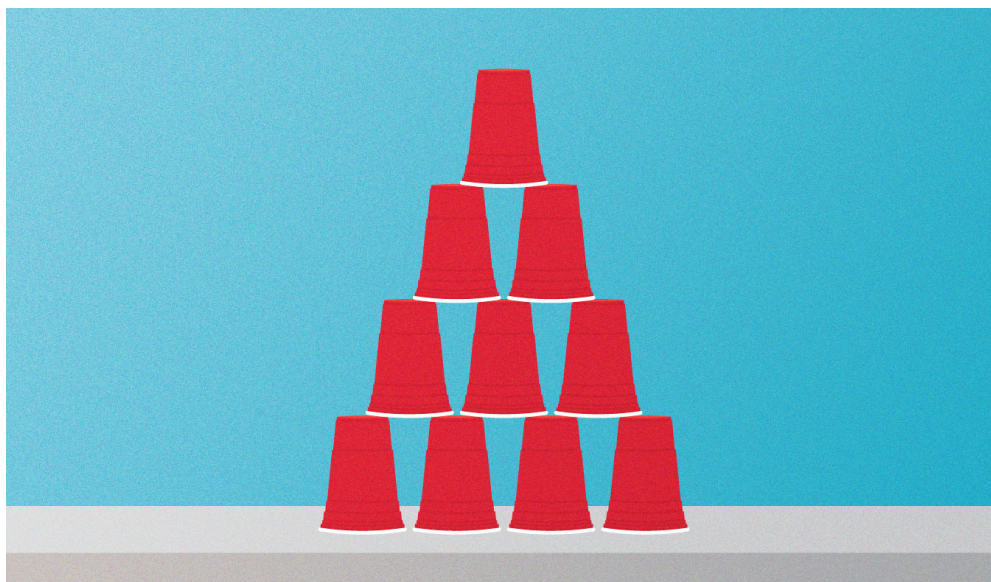
میانگین fitness جمعیت جدید برابر است با

$$\frac{40 + 30 + 49 + 21}{4} = \frac{140}{4} = 35.$$

بنابراین در این اجرا، میانگین fitness جمعیت جدید نسبت به جمعیت اولیه کاهش یافته است:

$$35 < 38.25.$$

۶. (۱۵ نمره) یک برج ساخته شده از لیوان‌ها داریم که به صورت لایه به لایه روی هم قرار گرفته‌اند. لیوان‌هایی که در پایین‌ترین ردیف قرار دارند مستقیماً روی میز هستند، و هر لیوان در ردیف‌های بالاتر دقیقاً روی دو لیوان زیر خود قرار دارد. در هر مرحله می‌توانیم لیوانی که حداکثر یک لیوان بالای آن است را انتخاب کرده و با ضربه توپ آن لیوان و تمامی لیوان‌هایی که مستقیم و غیرمستقیم بالایش هستند را فرو بریزیم. هدف بازی این است که با انجام کمترین تعداد ضربه، تعداد لیوان‌های فرو ریخته بیشتر از نیمی از کل لیوان‌ها شود.



(آ) فضای مسئله را طوری تعریف کنید که شامل حالت‌ها، کنش‌ها، هزینه کنش‌ها و حالت(های) هدف باشد.

(ب) وابستگان یک لیوان خاص را تعریف می‌کنیم: تمامی لیوان‌هایی که در صورت هدف قرار گرفتن آن لیوان فرو می‌ریزند. اگر تعداد وابستگان لیوان c را با $f(c)$ نمایش دهیم، تابع اکتشافی زیر را تعریف می‌کنیم:

$$h(S) = \frac{\text{cups}_{\text{needed}}}{\max_{c \in \text{targetable cups}(S)} f(c)}$$

که در آن صورت کسر برابر با کمینه تعداد لیوان‌هایی است که نیاز داریم سرنگون کنیم تا برنده شویم و مخارج آن بیشینه تعداد لیوان‌هایی است که می‌توانیم با یک ضربه توپ بیاندازیم. بررسی کنید که آیا این تابع قابل قبول و یک‌نوا هست یا نه و دلایل خود را توضیح دهید.

(ج) یک تابع اکتشافی دیگر ارائه دهید که قابل قبول باشد اما یک‌نوا نباشد. توضیح دهید چرا قابل قبول است ولی یک‌نوا نیست. (برای این بخش می‌توانید فرض کنید ارتفاع برج حداقل ۱۵ است.)

حل.

(آ) • **نمایش حالت:** حالت‌ها را می‌توان زیرمجموعه‌های مختلفی از لیوان‌هایی در نظر گرفت که هنوز نیفتاده‌اند. هرچند با قیدهای منطقی مثل «اگر لیوانی نیفتاده باشد، لیوان‌های زیر آن نیز نیفتاده‌اند» می‌توان تعداد حالت‌ها را محدود کرد، اما این کار ضروری نیست.

• **کنش‌ها:** هر کنش معادل پرتاب یک توپ به سمت یکی از لیوان‌هایی است که در حالت فعلی حداکثر یک لیوان رویی داشته باشند. به عبارت دیگر، هر کنش به معنای حذف لیوان هدف و تمام لیوان‌های بالای آن از مجموعه لیوان‌های فعال در زیرمجموعه فعلی است.

• **هزینه:** هزینه هر کنش برابر ۱ است.

• **حالت‌های هدف:** حالت‌هایی هستند که کمتر از نیمی از لیوان‌ها نیفتاده باشند.

(ب) حالتی را در نظر بگیرید که برچی به ارتفاع اولیه ۱۰ داریم. چهار ردیف پایینی سالم هستند، ولی در ردیف پنجم از پایین، لیوان سمت چپ و لیوان سمت راست حذف شده‌اند. در این حالت عملاً از هر ردیف بالایی ۲ لیوان کم شده، و به‌طور خاص دو ردیف بالایی کاملاً حذف شده‌اند. تعداد لیوان‌های اولیه ۵۵ است. تا کنون ۱۱ لیوان افتاده و باید ۱۷ لیوان دیگر نیز بیفتد. با بررسی می‌توان فهمید که بیشترین مقدار $f(c)$ در بین لیوان‌های قابل هدف‌گیری در این حالت برابر ۴ است. بنابراین تابع اکتشافی این حالت بیش از ۴ است. اما اگر دو لیوان از سمت چپ و دو لیوان از سمت راست پایین‌ترین ردیف

را حذف کنیم، $۷ + ۸ + ۴ + ۴ = ۲۳$ لیوان دیگر حذف می‌شوند و در ۴ حرکت به هدف می‌رسیم، و مسیر کوتاه‌تری تا هدف نیز وجود ندارد. در نتیجه تابع اکتشافی خوش‌بینانه نیست و نه یک‌نوا است و نه قابل قبول.

(ج) تابعی را در نظر بگیرید که برای همه حالت‌ها به جز حالت آغازین مقدار ۰ دارد و تنها برای حالت آغازین مقدار ۲. این تابع به وضوح کم‌برآورد می‌کند و قابل قبول است. اما از آنجا که هزینه هر کنش ۱ است و اختلاف تابع اکتشافی بین حالت آغازین و همسایه‌هایش ۲ می‌باشد، این تابع یک‌نوا نیست.

